

1 Problem 1 – Generator kongruencyjny

- Zaimplementuj liniowy generator kongruencyjny liczb pseudolosowych $x_1, x_2, \dots$: ($s_0 = 1$)
 $s_n = a s_{n-1} \bmod m$; $x_n = s_n / m$, $n = 1, 2, \dots$, gdzie $i \mid m = 37$, $a = 19$, $i \mid m = 2^{31} - 1$, $a = 39373$.
- Sprawdź empirycznie, jaki okres ma pierwszy z tych generatorów. • Narysuj 50 kolejnych liczb pseudolosowych j.w. jako punkty na odcinku jednostkowym. Narysuj histogram. Skomentuj rysunek.
- Narysuj 50 kolejnych par liczb pseudolosowych j.w. jako punkty na kwadracie jednostkowym. Skomentuj rysunek.

Rozwiązanie:

```
# zad 1
generator <- function(m, a, n = 50) {
  s <- 1
  liczby <- c()

  for (i in 1:n) {
    s <- (a * s) %% m
    liczby <- c(liczby, s / m)
  }
  return(liczby)
}

wynik1 <- generator(37, 19)
wynik2 <- generator(2 ^ 31 - 1, 39373)
```

Okres pierwszego z tych generatorów:

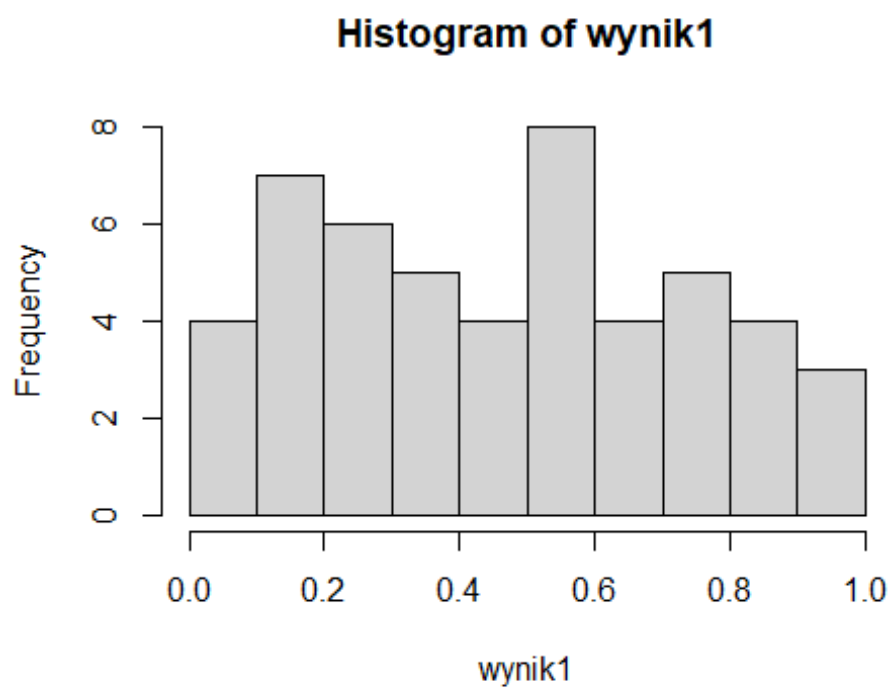
```
okres <- function(m, a) {
  s <- 1
  liczby <- c()

  while (TRUE) {
    s <- (a * s) %% m
    liczby <- c(liczby, s)
    if (any(duplicated(liczby))) {
      indeksy <- which(liczby == liczby[length(liczby)])
      return(indeksy[2] - indeksy[1])
    }
  }
}

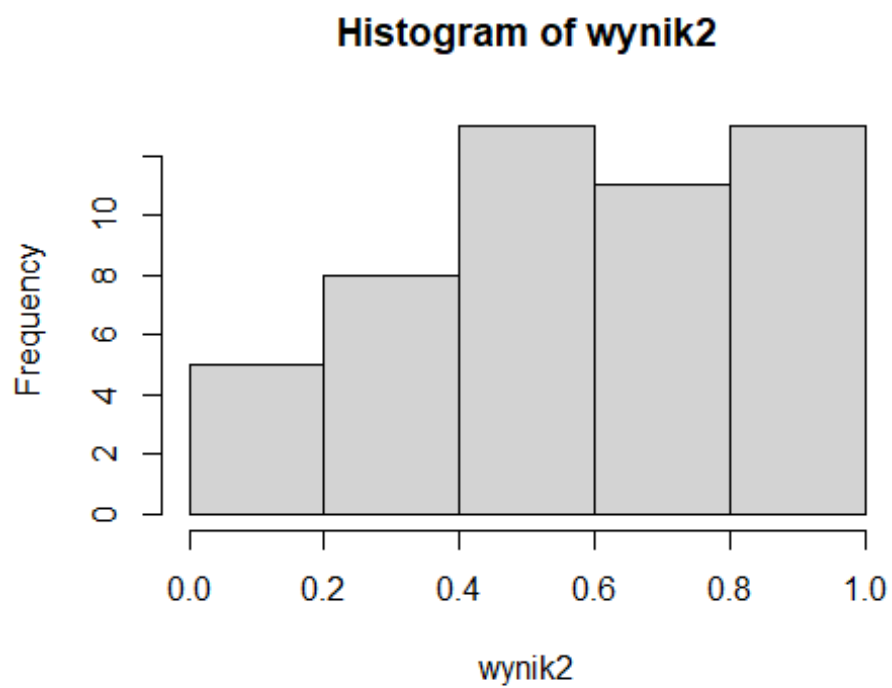
wynik3 <- okres(37, 19)
print(wynik3)

## [1] 36

# zad 3
hist(wynik1)
```



```
# zad 3.2  
hist(wynik2)
```

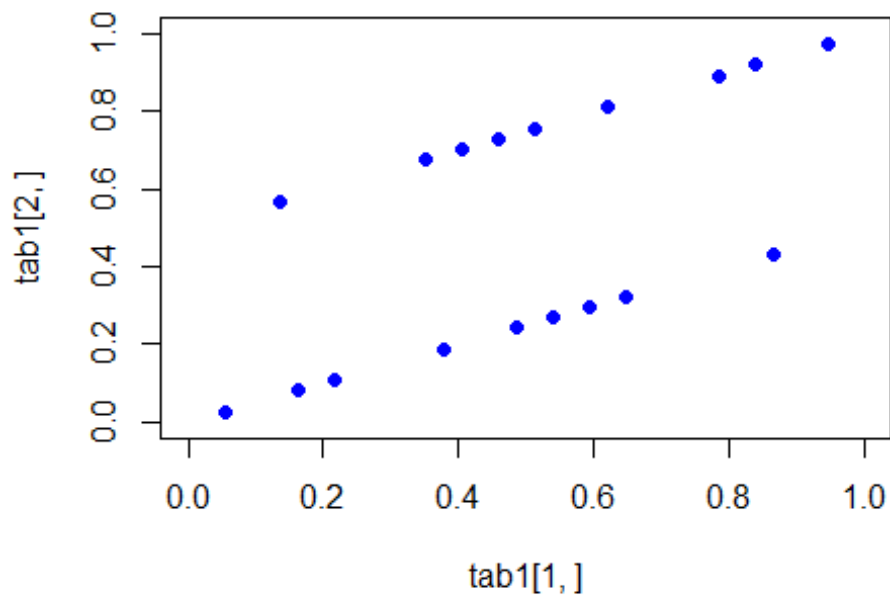


Histogram jest zbliżony do rozkładu jednostajnego, co jest pożądaną cechą generatorów pseudolosowych. Pewne odchylenia od rozkładu jednostajnego mogą być spowodowane ograniczoną długością ciągu danych.

zad 4

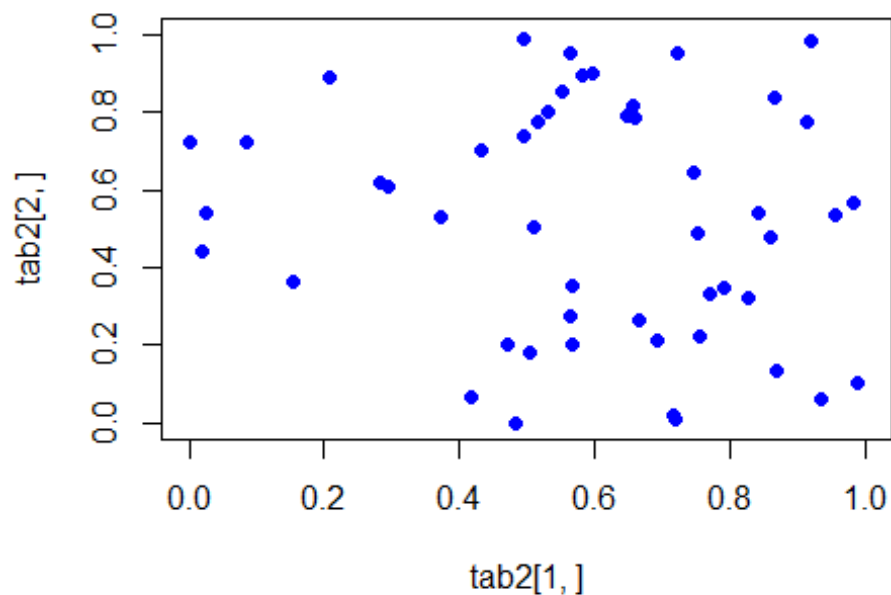
```
tab1 <- matrix(generator(37, 19, 100), 2, 500)
tab2 <- matrix(generator(2 ^ 31 - 1, 39373, 100), 2, 500)
```

```
plot(tab1[1,], tab1[2,], col = "blue", pch = 16, xlim = c(0, 1), ylim = c(0, 1))
```



Punkty układają się w linie co sugeruje że punkty ze sobą powiązane

```
plot(tab2[1,], tab2[2,], col = "blue", pch = 16, xlim = c(0, 1), ylim = c(0, 1))
```



Punkty nie są rozmieszczone równomiernie na całej powierzchni kwadratu. Widać pewne obszary, w których jest więcej punktów, co może sugerować istnienie korelacji między kolejnymi wartościami.

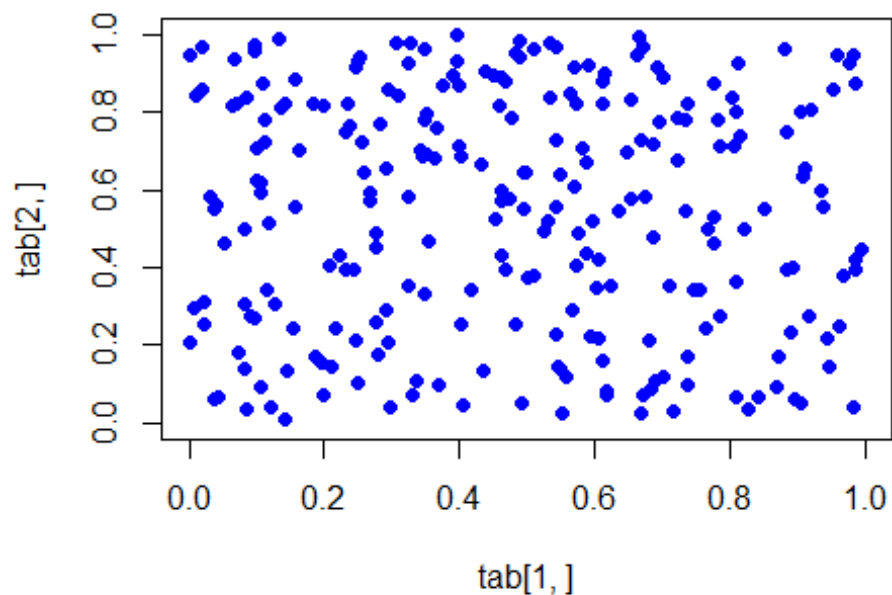
Narysuj 50 kolejnych par liczb pseudolosowych jako punkty na kwadracie jednostkowym dla standardowego generatora zaimplementowanego w pakiecie R.

- Czy otrzymane pary liczb pseudolosowych mają rozkład jednostajny na kwadracie $[0, 1]^2$? Zaproponuj, jak to weryfikować. W jaki sposób ta własność łączy się z własnością niezależności kolejnych liczb?
- Jak inicjować obliczenia, żeby za każdym razem otrzymywać ten sam ciąg 50-ciu par? Czy taka powtarzalność (replikowalność) jest sprzeczna z ideą symulacji pseudolosowych?

```
set.seed(131311)
```

```
tab <- matrix(runif(500), 2, 500)
```

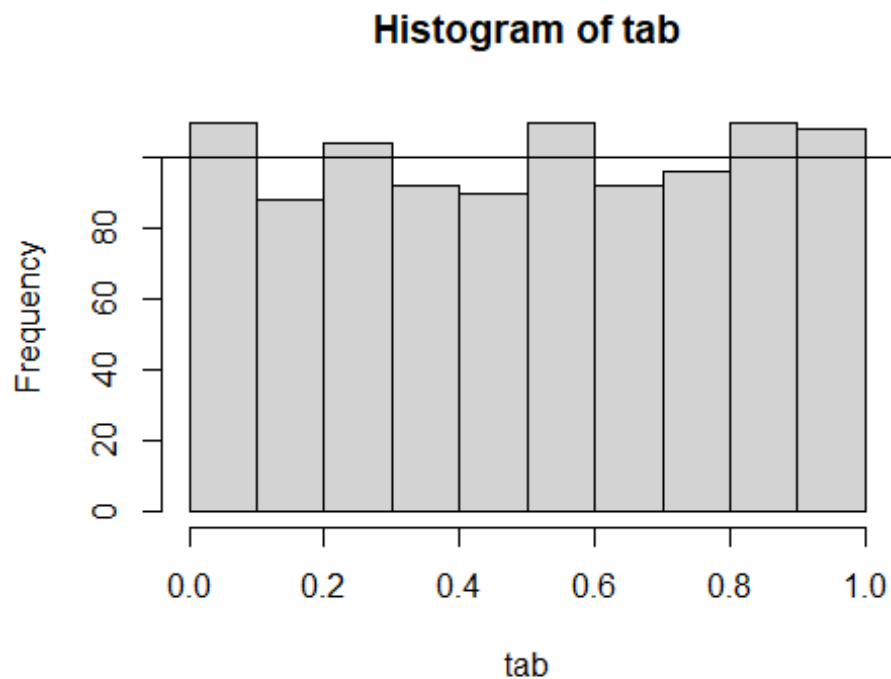
```
plot(tab[1,], tab[2,], col = "blue", pch = 16, xlim = c(0, 1), ylim = c(0, 1))
```



Rozkład

jednostajny można zweryfikować wizualnie, sprawdzając, czy punkty są równomiernie rozmieszczone na kwadracie. Rozkład jednostajny jest ściśle powiązany z niezależnością kolejnych liczb pseudolosowych. Gdyby liczby pseudolosowe były zależne, obserwowalibyśmy skupiska punktów na wykresie, co oznaczałoby, że pewne pary liczb pseudolosowych są bardziej prawdopodobne niż inne.

```
hist(tab)
abline(h = 100)
```



Innym sposobem jest sprawdzenie histogramu z nałożoną gęstością. W tym wypadku widać że faktycznie jest to rozkład jednostajny.

W R do inicjowania generatora liczb pseudolosowych używa się funkcji `set.seed()`. Podanie tej samej wartości ziarna (seed) przed wygenerowaniem liczb pseudolosowych zapewnia, że za każdym razem otrzymasz ten sam ciąg.

Przykład - `set.seed(131311)`

3 Problem 3 – Symulacja dyskretnych zmiennych losowych

- Napisz generator liczb losowych o rozkładzie dwupunktowym: $P(X = 1) = p$, $P(X = 0) = 1 - p$, gdzie $p \in [0, 1]$.
- Napisz generator liczb losowych dla rozkładu dwumianowego. Narysuj histogram dla próby (prostej) z tego rozkładu.
- Napisz generator dla rozkładu Poissona. Wykonaj odrębnie obliczenia przygotowawcze. Działanie generatora sprawdź poprzez analizę histogramów lub w inny sposób.

```
generator1 <- function(p) {
  temp <- runif(1, 0, 1)
  if (temp > p) {
    return(0)
  }
  else {
    return(1)
  }
}

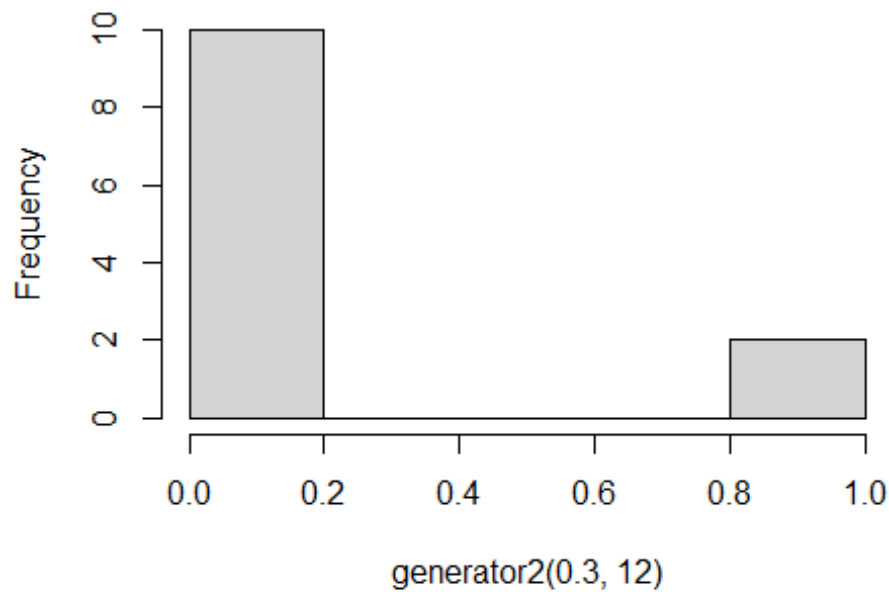
generator2 <- function(p, n) {
```

```

temp <- c()
for (i in 1:n) {
  temp = c(temp, generator1(p))
}
temp
}
hist(generator2(0.3, 12))

```

Histogram of generator2(0.3, 12)



```

generator3 <- function(lambda) {
  temp <- runif(1, 0, 1)
  k = 0
  p = exp(-lambda)
  F1 = p

  while (TRUE) {
    if (temp < F1) {
      return(k)
    }
    p = p * lambda / (k + 1)
    F1 = F1 + p
    k = k + 1
  }
}
#print(generator3(3))

generator4 <- function(p, n) {

```

```

temp <- c()
for (i in 1:n) {
  temp = c(temp, generator3(p))
}
temp
}
dane <- generator4(3, 12000)
hist(dane)
wyniki <- table(rpois(12000, 3))
lines(as.numeric(names(wyniki)), as.numeric(wyniki), type = "l", col =
"blue")

```

Histogram of dane

